

Problèmes Non Linéaires : Méthodes et Comparaison

Benaissa Roumeissa , Hamdane Salsabil

1^{er} octobre 2026

Table des matières

1	Introduction et définition des méthodes	3
2	Méthode de Gradient Descent	3
2.1	Gradient Descent à une variable	3
2.2	Gradient Descent à plusieurs variables	3
2.2.1	Algorithme du Gradient Descent	3
2.3	les avantages et les inconvénients :	4
3	Méthode de Newton-Raphson	4
3.1	Newton-Raphson pour une variable	4
3.2	Newton-Raphson pour plusieurs variables	4
3.2.1	Étapes détaillées de l'algorithme	5
3.3	Méthode du Gradient à Pas Fixe	5
4	Avantages et Inconvénients	5
4.1	Avantages et inconvénients de la méthode Newton-Raphson	5
4.2	Gradient à Pas Fixe	6
5	Comparaison entre Newton-Raphson et Gradient à pas fixe	6
6	Optimisation d'une cuve cylindrique (Vessel Problem) avec SciPy	6
6.1	Introduction	6
6.2	Description du problème	6
6.2.1	Variables de conception	6
6.2.2	Fonction objectif	7
6.2.3	Contraintes géométriques	7
6.2.4	Bornes des variables	7
6.3	Méthode de résolution avec SciPy	7
6.3.1	Principe de la méthode SLSQP	7
6.3.2	Avantages de la méthode	7
6.4	Point de départ et influence sur la solution	7
6.5	Étapes de résolution avec SciPy	7
6.6	Méthode de résolution avec Pyomo	8
6.6.1	Principe de la méthode	8
6.7	Étapes de résolution avec (Approximation)	8
6.8	La solution	8

7 Exemple de Machine Learning : Comparaison de MLP avec et sans optimisation non-linéaire

8

1 Introduction et définition des méthodes

Les problèmes non linéaires sont des problèmes où la relation entre les variables n'est pas linéaire. Dans ces problèmes, la solution est souvent **locale** et pas toujours **exacte ou globale**. Les méthodes numériques pour résoudre ces problèmes sont généralement **itératives** : elles commencent à partir d'un point initial et se déplacent progressivement vers la meilleure solution dans le voisinage de ce point.

Le succès de ces méthodes dépend fortement du **point initial**. Si le point de départ est loin de la solution optimale, le processus peut être plus lent ou même échouer. Parmi les méthodes principales : Newton-Raphson et Gradient à pas fixe, chacune ayant ses avantages et inconvénients selon la nature du problème.

2 Méthode de Gradient Descent

Gradient Descent est une méthode d'optimisation itérative utilisée pour trouver le minimum d'une fonction différentiable. Le principe consiste à mettre à jour la solution en suivant la direction opposée au gradient de la fonction, car cette direction correspond à la pente la plus forte en descente. À chaque itération, la mise à jour est donnée par :

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

où α est le pas d'apprentissage (learning rate) et $\nabla f(x_k)$ est le gradient de la fonction au point x_k . L'algorithme continue jusqu'à ce que le gradient devienne suffisamment petit, indiquant une convergence vers un minimum local ou global.

2.1 Gradient Descent à une variable

est une méthode d'optimisation visant à minimiser une fonction réelle $f(x)$, où $x \in \mathbb{R}$. L'idée est de mettre à jour la valeur de x en suivant la direction opposée à la dérivée de la fonction, puisque cette direction correspond à la plus forte descente. La mise à jour est donnée par :

$$x_{k+1} = x_k - \alpha f'(x_k),$$

où α est le pas d'apprentissage et $f'(x_k)$ représente la dérivée de f au point x_k .

2.2 Gradient Descent à plusieurs variables

permet de minimiser une fonction multivariée $f(\mathbf{x})$, où $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$. Dans ce cas, la dérivée est remplacée par le gradient $\nabla f(\mathbf{x})$, qui indique la direction de la plus forte montée. La mise à jour s'effectue alors dans la direction opposée :

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k),$$

où $\nabla f(\mathbf{x}_k)$ est le vecteur des dérivées partielles de f . L'algorithme converge lorsque la norme du gradient devient très petite.

2.2.1 Algorithme du Gradient Descent

1. **Initialisation** : choisir un point initial $x^{(0)}$ et un taux d'apprentissage $\alpha > 0$.

2. **Calcul du gradient** : calculer le gradient au point courant :

$$\nabla f(x^{(k)}).$$

3. **Mise à jour** : mettre à jour la variable selon la règle :

$$x^{(k+1)} = x^{(k)} - \alpha \nabla f(x^{(k)}).$$

4. **Sinon** : incrémenter k et revenir à l'étape 2.

2.3 le savantages et les incovénents :

Avantages :

- Simple à comprendre et à implémenter.
- Efficace pour les fonctions à haute dimension.
- Convergence rapide pour les fonctions convexes.
- Flexible et peut être adapté avec des variantes (SGD, mini-batch, momentum...).

Inconvénients :

- Convergence lente pour certaines fonctions avec des vallées étroites.
- Sensible au choix du learning rate α .
- Peut se bloquer dans un minimum local pour les fonctions non convexes.
- Nécessite que la fonction soit différentiable.

3 Méthode de Newton-Raphson

La méthode de Newton-Raphson est une méthode **itérative** utilisée pour trouver les racines d'une fonction ou pour minimiser une fonction. Elle peut être appliquée à une variable ou à plusieurs variables.

3.1 Newton-Raphson pour une variable

Pour une fonction $f(x)$ d'une seule variable, la méthode approxime f par une ligne tangente et calcule le zéro de cette tangente.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Étapes :

1. Choisir un point initial x_0 proche de la solution.
2. Calculer la dérivée $f'(x_k)$ et la valeur $f(x_k)$.
3. Calculer le nouveau point : $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$.
4. Répéter jusqu'à ce que $|x_{k+1} - x_k|$ soit inférieur à une tolérance ϵ .

3.2 Newton-Raphson pour plusieurs variables

Pour minimiser une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, on utilise l'approximation quadratique autour d'un point x_k :

$$f(x) \approx f(x_k) + \nabla f(x_k)^T (x - x_k) + \frac{1}{2} (x - x_k)^T H_k (x - x_k)$$

où :

- $\nabla f(x_k)$ est le gradient de f au point x_k .
 - $H_k = \nabla^2 f(x_k)$ est la Hessienne (matrice des dérivées secondes) de f au point x_k .
- En minimisant cette approximation quadratique, on obtient l'itération de Newton :

$$x_{k+1} = x_k - H_k^{-1} \nabla f(x_k)$$

3.2.1 Étapes détaillées de l'algorithme

1. **Initialisation** : choisir un point initial x_0 .
2. **Répéter jusqu'à convergence** :
 - (a) Calculer le gradient : $g_k = \nabla f(x_k)$.
 - (b) Calculer la Hessienne : $H_k = \nabla^2 f(x_k)$.
 - (c) Calculer le pas de Newton : $p_k = -H_k^{-1} g_k$.
 - (d) Mettre à jour le point : $x_{k+1} = x_k + p_k$.
3. Vérifier la condition d'arrêt, par exemple $\|g_k\| < \epsilon$ ou $\|x_{k+1} - x_k\| < \epsilon$.

Remarques :

- La méthode converge très rapidement si x_0 est proche du minimum.
- Elle nécessite le calcul de la Hessienne et son inversion, ce qui peut être coûteux pour des grandes dimensions.
- La convergence n'est pas garantie si le point initial est trop éloigné ou si la Hessienne n'est pas définie positive.

3.3 Méthode du Gradient à Pas Fixe

Descend le long de la direction du gradient :

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

4 Avantages et Inconvénients

4.1 Avantages et inconvénients de la méthode Newton-Raphson

Avantages :

- **Convergence rapide** : La méthode converge généralement **quadratiquement** si le point initial est proche du minimum.
- **Précision** : Elle peut fournir une solution très précise.
- **Utilisation flexible** : Applicable pour une variable ou plusieurs variables.

Inconvénients :

- **Sensibilité au point initial** : Si le point initial est trop éloigné, la méthode peut diverger.
- **Calcul de la Hessienne** : Pour plusieurs variables, il faut calculer et inverser la Hessienne, ce qui peut être coûteux en temps et mémoire.

- **Non garantie de convergence globale** : La méthode peut converger vers un minimum local et non le minimum global.
- **Dépendance à la régularité** : La Hessienne doit être définie et non singulière pour que l'inversion soit possible.

4.2 Gradient à Pas Fixe

- **Avantages** : Simple, pas besoin de Hessian, facile à implémenter.
- **Inconvénients** : Convergence lente, choix du pas critique.

5 Comparaison entre Newton-Raphson et Gradient à pas fixe

Critère	Newton-Raphson	Gradient à pas fixe
Convergence	Très rapide	Lente
Sensibilité au point initial	Élevée	Moyenne
Calcul requis	Gradient + Hessienne	Seulement gradient
Complexité	Élevée	Faible
Robustesse	Peut diverger	Plus stable
Solution	Minimum local	Minimum local

TABLE 1 – Comparaison simplifiée entre Newton-Raphson et Gradient à pas fixe

6 Optimisation d'une cuve cylindrique (Vessel Problem) avec SciPy

6.1 Introduction

Le Vessel Problem consiste à concevoir une cuve cylindrique en minimisant le coût total de fabrication tout en respectant certaines contraintes géométriques. C'est un exemple classique de problème d'optimisation non linéaire avec contraintes, souvent utilisé en ingénierie mécanique.

6.2 Description du problème

6.2.1 Variables de conception

Les variables principales sont :

- T_s : épaisseur de la calotte (couvercle)
- T_h : épaisseur du corps cylindrique (virole)
- R : rayon intérieur
- L : longueur de la cuve

6.2.2 Fonction objectif

La fonction objectif représente le coût total de fabrication :

$$\text{Cost} = 0.6224 T_s R L + 1.7781 T_h R^2 T_s^2 L + 19.84 T_s^2 R$$

6.2.3 Contraintes géométriques

— Limites minimales sur les épaisseurs en fonction du rayon :

$$T_s \geq 0.0193 \cdot R, \quad T_h \geq 0.00954 \cdot R$$

— Volume minimal requis :

$$V = \pi R^2 L \geq 129600000$$

6.2.4 Bornes des variables

$$T_s, T_h \geq 0.0625, \quad 10 \leq R, L \leq 200$$

6.3 Méthode de résolution avec SciPy

6.3.1 Principe de la méthode SLSQP

- La fonction objectif et les contraintes sont approximées localement par des fonctions quadratiques et linéaires.
- À chaque itération, un sous-problème simplifié est résolu pour obtenir un nouveau point.
- Le processus est répété jusqu'à convergence ou atteinte du nombre maximal d'itérations.

6.3.2 Avantages de la méthode

- Gestion efficace des contraintes non linéaires.
- Respect strict des bornes des variables.
- Adaptée aux problèmes d'ingénierie avec plusieurs variables continues.

6.4 Point de départ et influence sur la solution

- Le choix du point initial influence fortement la convergence et le résultat final.
- Une valeur initiale proche du minimum global permet une convergence rapide.
- Une valeur initiale éloignée peut conduire à un minimum local.
- Tester plusieurs points de départ et comparer les solutions améliore les chances de trouver le minimum global.

6.5 Étapes de résolution avec SciPy

1. Définir la fonction objectif représentant le coût total.
2. Définir les contraintes géométriques sous forme d'inégalités.
3. Définir les bornes pour chaque variable.

4. Choisir un point initial réalisable.
5. Lancer la méthode SLSQP via SciPy.
6. Suivre la convergence de la fonction objectif et l'évolution des variables.

6.6 Méthode de résolution avec Pyomo

6.6.1 Principe de la méthode

Le problème de conception du cylindre est formulé comme un problème d'optimisation non linéaire avec contraintes, dans lequel la fonction objectif ainsi que certaines contraintes géométriques sont non linéaires. Dans l'environnement étudié, la résolution directe par des solveurs NLP tels que IPOPT n'étant pas possible, une approche d'approximation a été adoptée. Cette méthode consiste à discrétiser l'espace des variables continues et à approximer les fonctions non linéaires par une combinaison convexe de valeurs pré-calculées. Le problème ainsi transformé devient un modèle de programmation linéaire mixte (MILP), modélisé à l'aide de Pyomo et résolu efficacement par le solveur Gurobi. Cette approche permet d'obtenir une solution approchée tout en respectant les contraintes physiques et industrielles du problème.

6.7 Étapes de résolution avec (Approximation)

1. Formulation du problème comme un problème d'optimisation non linéaire avec contraintes.
2. Discrétisation des variables continues et calcul des valeurs de la fonction objectif et des contraintes.
3. Approximation des fonctions non linéaires par une combinaison convexe afin d'obtenir un modèle linéaire mixte.
4. Modélisation du problème sous Pyomo et résolution du modèle approximé à l'aide du solveur Gurobi.

6.8 La solution

La qualité de la solution obtenue par l'approche d'approximation dépend de la finesse de la discrétisation des variables continues. Plus le maillage des points de discrétisation est dense, plus l'approximation des fonctions non linéaires est précise, conduisant à une solution proche de l'optimum réel du problème non linéaire initial. Cependant, un maillage trop fin augmente la taille du modèle MILP et le temps de calcul. Ainsi, un compromis entre précision et efficacité de calcul est nécessaire pour garantir une solution à la fois faisable et de qualité.

7 Exemple de Machine Learning : Comparaison de MLP avec et sans optimisation non-linéaire

Dans cet exemple, nous utilisons un **réseau de neurones** (`MLPClassifier`) pour classer des avis textuels en positifs ou négatifs.

- **Données** : une petite liste de textes avec leurs labels (1 = positif, 0 = négatif), transformés en vecteurs TF-IDF.

- **Modèle avec optimisation non-linéaire :** le MLP est entraîné normalement avec plusieurs itérations pour réduire la fonction de perte (Loss), ce qui améliore la précision.
- **Modèle sans optimisation non-linéaire :** le MLP est entraîné de manière minimale (1 seule itération sur un petit échantillon), ce qui empêche l'amélioration réelle de la Loss.

Résultats : Le modèle avec optimisation non-linéaire montre une *accuracy* plus élevée et un *log loss* plus faible par rapport au modèle sans optimisation. Cela illustre l'importance de l'optimisation non-linéaire dans l'apprentissage des réseaux de neurones.